

ใบงานเรื่อง โครงสร้างของต้นไม้ AVL-Tree และ B-Tree

จัดทำโดย

นาย ศิริพงศ์ บุญประสิทธิ์

รหัส ENGCE124_SEC2

เสนอ

อาจารย์ ปิยพล ยืนยงสถาวร

ใบงานนี้เป็นส่วนหนึ่งของวิชา

ENGCE124

(Data Structures and Algorithms)

หลักสูตร วศ.บ.วิศวกรรมคอมพิวเตอร์

สาขาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา ภาควิชาฯ เชียงใหม่

ภาคเรียนที่ 1 ปีการศึกษา 2569

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อศึกษาความรู้เกี่ยวกับโครงสร้างข้อมูลประเภทต้นไม้ (Tree Data Structure) ซึ่งเป็นหนึ่งในโครงสร้างข้อมูลที่มีความสำคัญอย่างยิ่งในศาสตร์ด้านวิทยาการคอมพิวเตอร์และการพัฒนาซอฟต์แวร์ โดยเฉพาะอย่างยิ่ง AVL-Tree และ B-Tree ซึ่งเป็นโครงสร้างต้นไม้ ที่ถูกออกแบบมาเพื่อเพิ่มประสิทธิภาพในการจัดเก็บ ค้นหา เพิ่ม และลบข้อมูลให้สามารถทำงานได้อย่างรวดเร็วและมีประสิทธิภาพ

เนื้อหาในรายงานประกอบด้วยโครงสร้าง และหลักการทำงานของ AVL-Tree และ B-Tree รวมถึงตัวอย่างการสร้างต้นไม้ การปรับสมดุลของต้นไม้ การเพิ่มข้อมูล การลบข้อมูล และการค้นหาข้อมูล เพื่อให้เข้าใจถึงกระบวนการทำงานและข้อแตกต่างของโครงสร้างต้นไม้ทั้งสองประเภท ตลอดจนสามารถนำความรู้ที่ได้รับไปประยุกต์ใช้ในการออกแบบระบบฐานข้อมูลและโปรแกรมคอมพิวเตอร์ได้อย่างเหมาะสม

ผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์แก่ผู้อ่านที่สนใจศึกษาเกี่ยวกับโครงสร้างข้อมูล และช่วยให้เกิดความเข้าใจเกี่ยวกับ AVL-Tree และ B-Tree มากยิ่งขึ้น หากมีข้อผิดพลาดประการใด ผู้จัดทำขอน้อมรับคำแนะนำและข้อเสนอแนะเพื่อนำไปปรับปรุงให้ดียิ่งขึ้น

ผู้จัดทำ

นาย ศิริพงศ์ บุญประสิทธิ์

วันที่ 6 ตุลาคม 2569

สารบัญ

คำนำ	2
ความรู้พื้นฐานของต้นไม้ (Tree)	4
Binary Search Tree (BST)	4
Adelson-Velsky and Landis Tree (AVL-Tree)	5
การสร้าง AVL-Tree	5
การปรับสมดุล AVL-Tree (Rotation)	6
การเพิ่ม ลบ และค้นหาใน AVL-Tree	7
Balanced Tree (B-Tree)	8
โครงสร้างของ B-Tree	8
การเพิ่มข้อมูลใน B-Tree	8
การลบข้อมูลใน B-Tree	8
การค้นหาข้อมูลใน B-Tree	8
เปรียบเทียบ AVL-Tree และ B-Tree	9
สรุปเนื้อหาทั้งหมด	9

ความรู้พื้นฐานของต้นไม้ (Tree)

Tree เป็นโครงสร้างข้อมูลแบบไม่เชิงเส้น (Non-Linear Data Structure) ที่ประกอบด้วยโหนด (Node) หลายโหนดเชื่อมโยงกันเป็นลำดับชั้น (Hierarchy) โดยมีโหนดเริ่มต้นเรียกว่า Root Node และแตกแขนงออกเป็นโหนดย่อยต่าง ๆ

องค์ประกอบสำคัญได้แก่

- Root Node คือโหนดแรกของต้นไม้
- Parent Node คือโหนดแม่
- Child Node คือโหนดลูก
- Sibling Node คือโหนดที่มี
- Parent เดียวกัน
- Leaf Node คือโหนดปลายสุดที่ไม่มีลูก
- Subtree คือต้นไม้ย่อยของโหนดใดโหนดหนึ่ง

โครงสร้างต้นไม้ถูกนำมาใช้ในการจัดเก็บข้อมูลจำนวนมากเพื่อให้ค้นหาได้รวดเร็วกว่าการเก็บแบบลำดับในอาร์เรย์หรือรายการเชื่อมโยง

Binary Search Tree (BST)

ก่อนศึกษา AVL-Tree จำเป็นต้องเข้าใจ BST ก่อน เพราะ AVL-Tree เป็นการพัฒนาจาก BST โดยตรง โครงสร้างของ BST คือ Binary Tree ที่ทุกโหนดมีลูกได้ไม่เกิน 2 โหนด และมีเงื่อนไขว่าค่าของโหนดในต้นไม้ย่อยด้านซ้ายต้องน้อยกว่าค่าของ Root และค่าของต้นไม้ย่อยด้านขวาต้องมากกว่าหรือเท่ากับ Root เขียนได้เป็น

$$TL < R \leq TR$$

ตัวอย่างการสร้าง BST ชุดข้อมูล 22, 8, 7, 25, 21, 29, 42, 28

การสร้าง นำ 22 เป็น Root $8 < 22$ ไปซ้าย

$7 < 8$ ไปซ้ายของ 8 , $25 > 22$ ไปขวา , $21 > 8$ แต่ < 22 , $29 > 25$, $42 > 29$, $28 > 25$ แต่ < 29

ผลลัพธ์จะได้ต้นไม้ BST ตามลำดับการแทรกข้อมูล

การค้นหาใน BST หลักการทำงาน เริ่มเปรียบเทียบกับ Root ถ้าพบ จบการค้นหา ถ้าน้อยกว่า ไปทางซ้าย ถ้ามากกว่า ไปทางขวา ทำซ้ำจนกว่าจะพบข้อมูลหรือถึงปลายต้นไม้

ข้อดีคือไม่ต้องตรวจสอบทุกโหนดเหมือน Sequential Search

การเพิ่มข้อมูลใน BST การเพิ่มข้อมูลใช้หลักการเดียวกับการสร้างต้นไม้ ตัวอย่าง เพิ่มค่า 35 หรือ 35 > Root จึงเลื่อนไปทางขวาเรื่อย ๆ จนถึงตำแหน่งว่าง แล้วสร้างโหนดใหม่เพิ่มเข้าไป

การลบข้อมูลใน BST มี 3 กรณี กรณีที่ 1 ลบ Leaf Node เช่น 1 4 20 35 สามารถลบออกได้ทันทีเพราะไม่มีลูก กรณีที่ 2 ลบโหนดที่มีลูกเพียงด้านเดียว ให้ดึงลูกขึ้นมาแทนที่โหนดเดิม ตัวอย่าง 5->4 เมื่อลบ 5 ตัวข้อมูล 4 จะขึ้นมาเป็นโหนดแทนทันที กรณีที่ 3 ลบโหนดที่มีลูกสองด้าน ต้องเลือกค่ามากสุดในต้นไม้ย่อยซ้ายหรือค่าน้อยที่สุดต้นไม้ย่อยขวาขึ้นมาแทนค่าที่ถูกลบเพื่อรักษาคุณสมบัติ BST ไว้

Adelson-Velsky and Landis Tree (AVL-Tree)

ความหมายของ AVL-Tree AVL Tree เป็น Binary Search Tree ชนิดหนึ่งที่รักษาความสมดุลของความสูงของต้นไม้เอาไว้เสมอ ถูกคิดค้นโดย Adelson-Velskii , Landis จึงเรียกรวมว่า AVL Tree

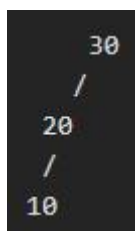
โครงสร้างของ AVL-Tree AVL Tree จะมีคุณสมบัติเพิ่มจาก BST คือ สำหรับทุกโหนด $Height(Left) - Height(Right)$ ต้องมีค่าเป็น -1, 0, +1 เท่านั้น เรียกว่า Balance Factor ถ้ามากกว่า 1 หรือน้อยกว่า -1 จะถือว่าไม่สมดุลและต้องปรับต้นไม้ทันที

หลักการของ AVL-Tree แนวคิดสำคัญคือ รักษาความสูงของต้นไม้ให้สมดุลอยู่เสมอ เพราะถ้าต้นไม้เอียงมากเกินไป BST จะกลายเป็นลักษณะคล้าย Linked List ทำให้ประสิทธิภาพลดลง AVL Tree จึงใช้เทคนิค Rotation หรือการหมุนต้นไม้เพื่อคงให้ระดับความสูงกลับมาสมดุลอีกครั้ง

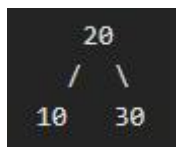
การสร้าง AVL-Tree

หลักการสร้าง AVL-Tree AVL-Tree มีวิธีสร้างเหมือน Binary Search Tree ทุกประการ แต่หลังจากเพิ่มข้อมูลทุกครั้ง จะต้องตรวจสอบค่าความสมดุล (Balance Factor) ของทุกโหนดที่ได้รับผลกระทบ หากพบว่าค่าความสมดุลเกินช่วง -1 ถึง +1 จะต้องทำการ Rotation เพื่อปรับต้นไม้ให้สมดุลทันที ตัวอย่างการสร้าง AVL-Tree เพิ่มข้อมูลตามลำดับ 30, 20, 10

ชั้นที่ 1 30, ชั้นที่ 2 30->20, ชั้นที่ 3 30->20->10



คำนวณ Balance Factor BF (30) = 2 ต้นไม้เสียสมดุล จึงต้องหมุนแบบ Right Rotation ผลลัพธ์คือ กลับมาสมดุลอีกครั้ง

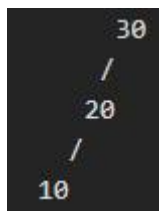


การปรับสมดุล AVL-Tree (Rotation)

เมื่อเกิดความไม่สมดุล AVL-Tree จะใช้การหมุนต้นไม้ (Rotation) เพื่อปรับโครงสร้าง มีทั้งหมด 4 กรณี

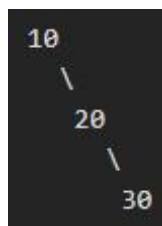
5.1 Single Right Rotation (SRR) เกิดกรณี Left-Left (LL) หมุนขวา

ใช้เมื่อเพิ่มข้อมูลทางซ้ายต่อเนื่องหลายระดับ



5.2 Single Left Rotation (SLR) เกิดกรณี Right-Right (RR) หมุนซ้าย

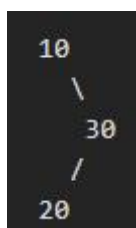
ใช้เมื่อเพิ่มข้อมูลทางขวาต่อเนื่องหลายระดับ



5.3 Double Right Rotation (DRR) เกิดกรณี Left-Right (LR) หมุนซ้ายที่ 10 จากนั้นหมุนขวาที่ 30 เรียกว่า Double Rotation



5.4 Double Left Rotation (DLR) เกิดกรณี Right-Left (RL) หมุนขวาที่ 30 จากนั้นหมุนซ้ายที่ 10 ผลคือกลับมาสมดุลอีกครั้ง



การเพิ่ม ลบ และค้นหาใน AVL-Tree

การเพิ่มข้อมูล (Insertion) ขั้นตอน

1. เพิ่มข้อมูลแบบ BST
2. ตรวจสอบ Balance Factor
3. ถ้าสมดุลไม่เกิน ± 1 จบการทำงาน
4. ถ้าเกิน ± 1 ให้ทำ Rotation ที่เหมาะสม
5. ปรับความสูงใหม่ทุกโหนดที่เกี่ยวข้อง

การลบข้อมูล (Deletion) การลบใน AVL-Tree ทำเหมือน BST แต่หลังลบเสร็จต้องตรวจสอบ Balance Factor อีกครั้ง ถ้าไม่สมดุลต้องทำ Rotation

การค้นหาข้อมูล (Searching) เหมือน BST ทุกประการ ขั้นตอนเริ่มจาก Root เปรียบเทียบข้อมูล น้อยกว่า ไป ซ้าย มากกว่า ไป ขวา พบข้อมูล จบ เนื่องจากต้นไม้สมดุลอยู่เสมอความซับซ้อนของการค้นหา $O(\log n)$ ซึ่งเร็วกว่ากรณี BST เสียสมดุลที่อาจกลายเป็น $O(n)$

Balanced Tree (B-Tree)

ความหมายของ B-Tree B-Tree เป็นโครงสร้างต้นไม้สำหรับ การค้นหาข้อมูลที่ได้รับการออกแบบเพื่อรองรับข้อมูลจำนวนมาก โดยมักใช้ในระบบฐานข้อมูลและระบบจัดเก็บข้อมูลบนดิสก์ แตกต่างจาก AVL-Tree ตรงที่ 1 โหนดเก็บได้หลายค่า 1 โหนดมีลูกได้หลายโหนด ความสูงของต้นไม้ต่ำมาก เหมาะกับฐานข้อมูลขนาดใหญ่

โครงสร้างของ B-Tree

คุณสมบัติสำคัญทุกโหนดมีข้อมูลเรียงลำดับทุกใบอยู่ในระดับเดียวกัน โหนดหนึ่งมีลูกได้หลายโหนด รักษาสมดุลตลอดเวลา

การเพิ่มข้อมูลใน B-Tree

กรณีโหนดยังไม่เต็ม เพิ่มข้อมูลแล้วเรียงลำดับ , กรณีโหนดเต็ม หากเพิ่มแล้วเกินจำนวน Key ที่กำหนด ต้อง Split

การลบข้อมูลใน B-Tree

การลบซับซ้อนกว่า AVL-Tree หลักการคือ ลบข้อมูลออก ตรวจสอบว่าจำนวน Key ต่ำกว่าค่าที่กำหนดหรือไม่ ถ้าต่ำเกินไปต้อง Borrow ขอยืม Key จากพี่น้อง Sibling Node หากมีข้อมูลเพียงพอหรือ Merge รวมโหนดเข้าด้วยกันเมื่อต้นไม้ขาดข้อมูลเกินไปผลลัพธ์คือยังคงรักษาความสมดุลของต้นไม้ได้

การค้นหาข้อมูลใน B-Tree

ขั้นตอน เริ่มที่ Root เปรียบเทียบกับทุก Key ในโหนด ถ้าพบ จบการค้นหา ถ้าไม่พบ จะลงไปยัง Subtree ที่เหมาะสม ทำซ้ำจนพบข้อมูล

เปรียบเทียบ AVL-Tree และ B-Tree

หัวข้อ	AVL-Tree	B-Tree
จำนวนลูกต่อโหนด	ไม่เกิน 2	หลายโหนด
ประเภท	Binary Search Tree	Multi-way Search Tree
การรักษาสมดุล	Rotation	Split/Merge
ความสูงต้นไม้	มากกว่า B-Tree	ต่ำกว่า
การค้นหา	$O(\log n)$	$O(\log n)$
การเพิ่มข้อมูล	Rotation	Split
การลบข้อมูล	Rotation	Borrow/Merge
การใช้งาน	โครงสร้างข้อมูลในหน่วยความจำ	ฐานข้อมูลและระบบไฟล์
เหมาะกับ	ข้อมูลขนาดกลาง	ข้อมูลขนาดใหญ่มาก

สรุปเนื้อหาทั้งหมด

AVL-Tree และ B-Tree

เป็นโครงสร้างข้อมูลประเภทต้นไม้เพื่อการค้นหาที่ได้รับการพัฒนามาจากแนวคิดของ Binary Search Tree เพื่อแก้ปัญหากลี่ยสมดุลของต้นไม้ โดย AVL-Tree ใช้วิธี Rotation เพื่อรักษาความสมดุลของความสูง ทำให้สามารถค้นหา เพิ่ม และลบข้อมูลได้อย่างมีประสิทธิภาพที่ระดับ $O(\log n)$ ส่วน B-Tree

ออกแบบมาเพื่อจัดเก็บข้อมูลจำนวนมาก

โดยอนุญาตให้หนึ่งโหนดเก็บข้อมูลได้หลายค่าและมีลูกได้หลายโหนด ใช้กระบวนการ Split, Borrow และ Merge ในการรักษาสมดุลของต้นไม้ จึงเหมาะสำหรับงานฐานข้อมูลและระบบจัดเก็บข้อมูลขนาดใหญ่ ทั้งสองโครงสร้างช่วยลดเวลาในการเข้าถึงข้อมูลและเป็นพื้นฐานสำคัญของระบบสารสนเทศสมัยใหม่